

Final Project AOL – Advanced Linear Algebra

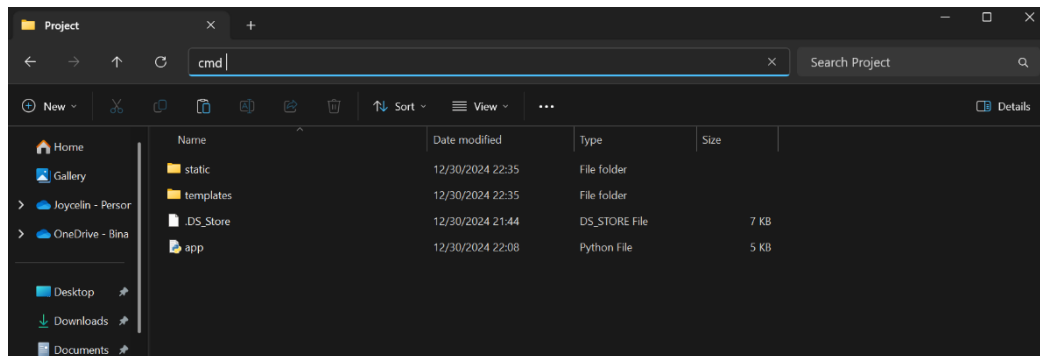
Documentation for Users and Developers

Group 10

- Ann Abigail Halim (2702260943)
- Hania Nayma Zahra (2702375683)
- Joycelin (2702213713)
- Richelle Marvela (2702208543)

How to Start the Application:

- 1) Unzip the folder named “FINAL_AOL_ADVLLINALG”.
- 2) Open the “Project” folder.
- 3) Click on the address bar at the top of the folder, open the command prompt (cmd) by type “cmd” and enter.

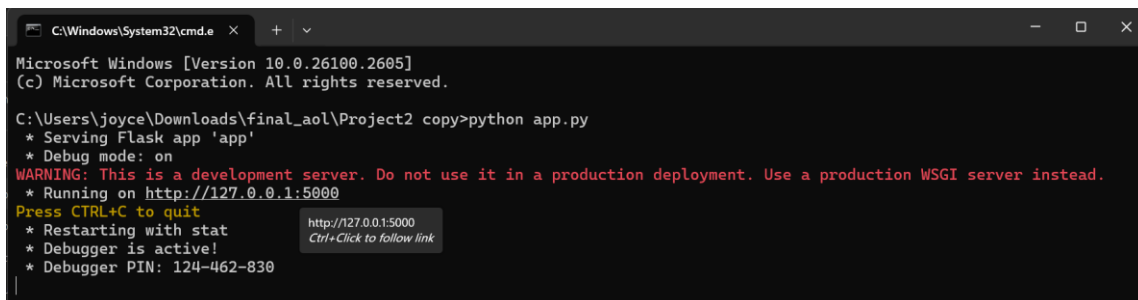


- 4) Type “python app.py”

If not working, install from command prompt:

- flask by typing “pip install flask”
- numpy library by typing “pip install numpy”
- scipy library by typing “pip install scipy”

- 5) Copy the local host <http://127.0.0.1:5000> and paste on the browser, or just CTRL+Click the link from command prompt.

A screenshot of a Windows Command Prompt window titled 'C:\Windows\System32\cmd.e'. The text shows the execution of 'python app.py' and the resulting output: 'Microsoft Windows [Version 10.0.26100.2605] (c) Microsoft Corporation. All rights reserved. C:\Users\joyce\Downloads\final_aol\Project2 copy>python app.py * Serving Flask app 'app' * Debug mode: on WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://127.0.0.1:5000 Press CTRL+C to quit * Restarting with stat http://127.0.0.1:5000 Ctrl+Click to follow link * Debugger is active! * Debugger PIN: 124-462-830'.

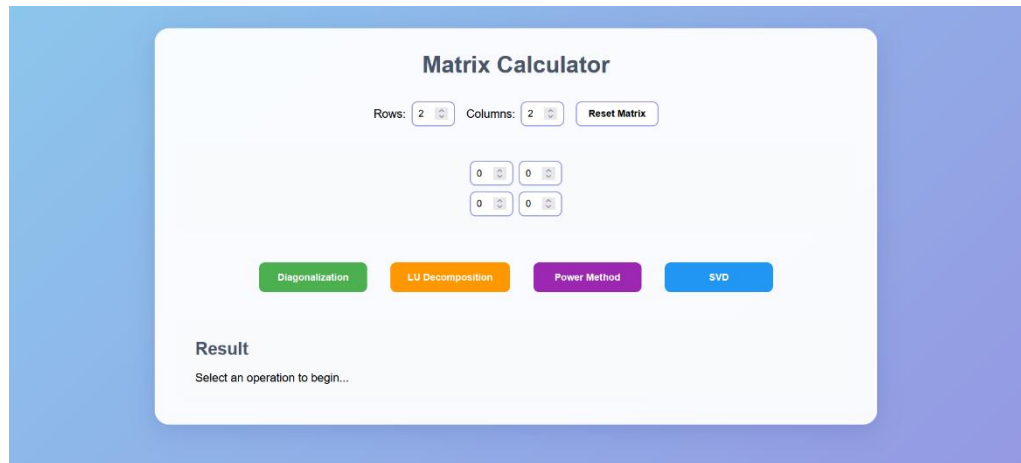
Documentation for Users

1. Diagonalizability Check

A. How to Use the Application and Test Case

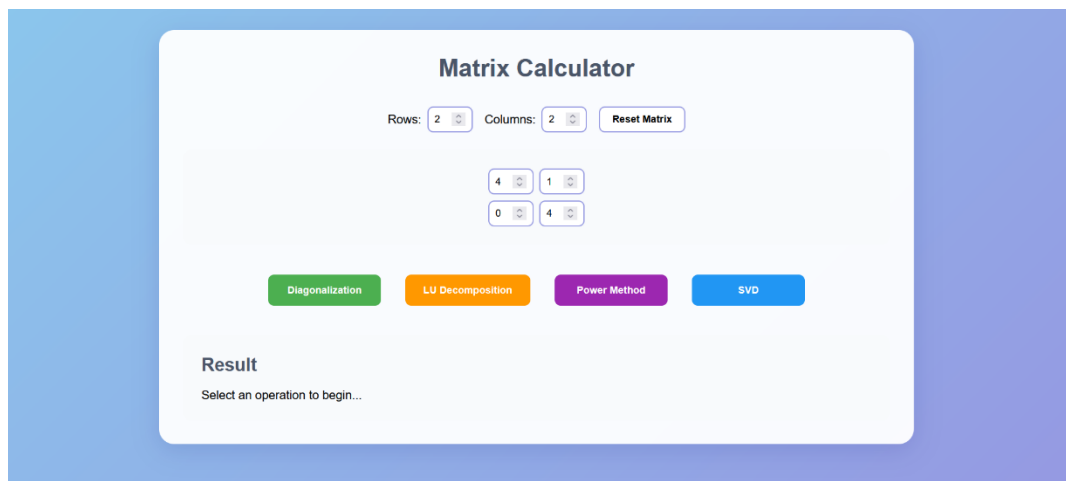
a) Input

- 1) Select the matrix size from the dropdown or type the size you wanted, then click “Reset Matrix”.



The screenshot shows the 'Matrix Calculator' interface. At the top, it says 'Matrix Calculator'. Below that, there are input fields for 'Rows' and 'Columns', both set to '2', and a 'Reset Matrix' button. In the center, there is a 2x2 grid of input fields for matrix elements, all containing '0'. Below the grid are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). At the bottom, there is a 'Result' section with the text 'Select an operation to begin...'.

- 2) Enter the matrix elements.



The screenshot shows the 'Matrix Calculator' interface with the matrix elements updated. The 'Rows' and 'Columns' fields remain at '2', and the 'Reset Matrix' button is still present. The 2x2 grid of input fields now contains the values 4, 1, 0, and 4. The buttons 'Diagonalization', 'LU Decomposition', 'Power Method', and 'SVD' are still visible. The 'Result' section at the bottom still shows 'Select an operation to begin...'.

- 3) Choose the operation we would like to perform. For this case, choose “Diagonalization”.
- 4) The result will shown.

b) Output

For this test case, we use the matrix $\begin{bmatrix} 4.0 & 1.0 \\ 0.0 & 4.0 \end{bmatrix}$.

The screenshot shows the 'Matrix Calculator' interface. At the top, the title 'Matrix Calculator' is centered. Below it, there are input fields for 'Rows' (set to 2) and 'Columns' (set to 2), along with a 'Reset Matrix' button. The matrix input area shows a 2x2 grid with values 4, 1, 0, and 4. Below the input area, there are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). The 'Result' section is displayed below the buttons, showing the following output:

```
Result
Diagonalization Results
Diagonalizable: false
Eigenvalues:
4, 4
Eigenvectors:
[ 1.0000 -1.0000 ]
[ 0.0000  0.0000 ]
```

The result shows that:

1) Diagonalizable: false

Shows that the matrix $\begin{bmatrix} 4.0 & 1.0 \\ 0.0 & 4.0 \end{bmatrix}$ is not diagonalizable.

2) Eigenvalues: $[4.0, 4.0]$

Shows that both eigenvalues are 4 (repeated).

3) Eigenvectors: $\begin{bmatrix} 1.0000 & -1.0000 \\ 0.0000 & 0.0000 \end{bmatrix}$

Shows the eigenvector corresponding to the eigenvalue 4 but in different direction.

We use another test case, with the matrix $\begin{bmatrix} 5.0 & 4.0 \\ 1.0 & 2.0 \end{bmatrix}$.

The screenshot shows the 'Matrix Calculator' interface. At the top, the title 'Matrix Calculator' is centered. Below it, there are input fields for 'Rows' (set to 2) and 'Columns' (set to 2), along with a 'Reset Matrix' button. The matrix input area shows a 2x2 grid with values 5, 4, 1, and 2. Below the input area, there are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). The 'Result' section is displayed below the buttons, showing the following output:

```
Result
Diagonalization Results
Diagonalizable: true
Eigenvalues:
6, 1
Eigenvectors:
[ 0.9701 -0.7071 ]
[ 0.2425  0.7071 ]
```

The result shows that:

1) Diagonalizable: True

Shows that the matrix $\begin{bmatrix} 5.0 & 4.0 \\ 1.0 & 2.0 \end{bmatrix}$ is diagonalizable. This means that it can be represented as the product PDP^{-1} , where D is a diagonal matrix containing the eigenvalues, and P is the matrix of eigenvectors.

2) Eigenvalues: [6.0, 1.0]

Shows that the eigenvalues are 6.0 and 1.0. These values represent the scalar values that satisfy the equation $A \cdot v = \lambda \cdot v$, where v is an eigenvector corresponding to each eigenvalue. The eigenvalue 6 is the dominant one, meaning it has the larger magnitude.

3) Eigenvectors: $\begin{bmatrix} 0.9701 & -0.7071 \\ 0.2425 & 0.7071 \end{bmatrix}$

These are the eigenvectors corresponding to the eigenvalues 6.0 and 1.0. The first column is the eigenvector based on the eigenvalue 6.0, while the second column is based on with the eigenvalue 1.0. The two eigenvectors are linearly independent, which proves that the matrix is diagonalizable, and this also means that this matrix can be decomposed into a diagonal matrix.

B. Explanation

a) Diagonalizability

A matrix is diagonalizable if it can be written as $A = PDP^{-1}$, where P is a matrix of eigenvectors, and D is a diagonal matrix of eigenvalues. A matrix is diagonalizable if and only if it has enough linearly independent eigenvectors to form the matrix P .

b) Eigenvalues and Eigenvectors

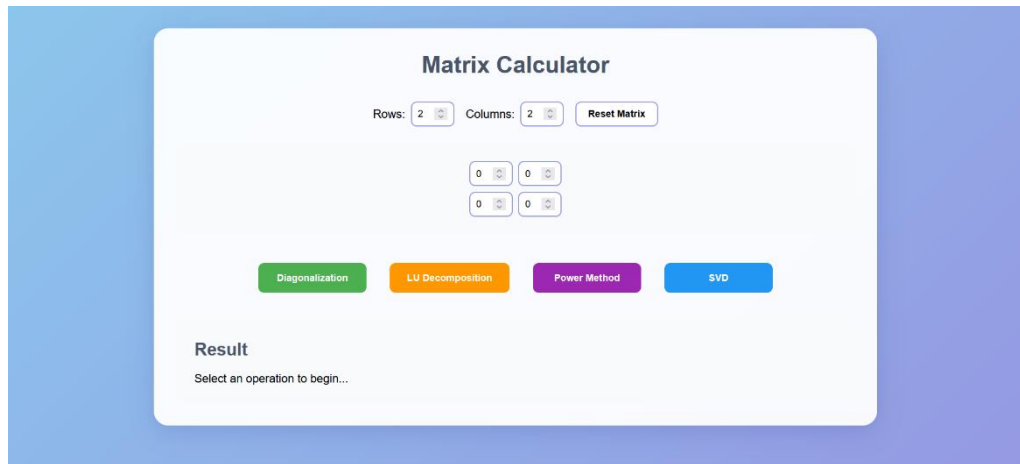
An eigenvalue λ is a scalar such that $Av = \lambda v$, where A is a matrix and v is a non-zero vector (the eigenvector) associated with λ . The matrix's eigenvectors form the basis for the diagonalization.

2. LU Decomposition

A. How to Use the Application and Test Case

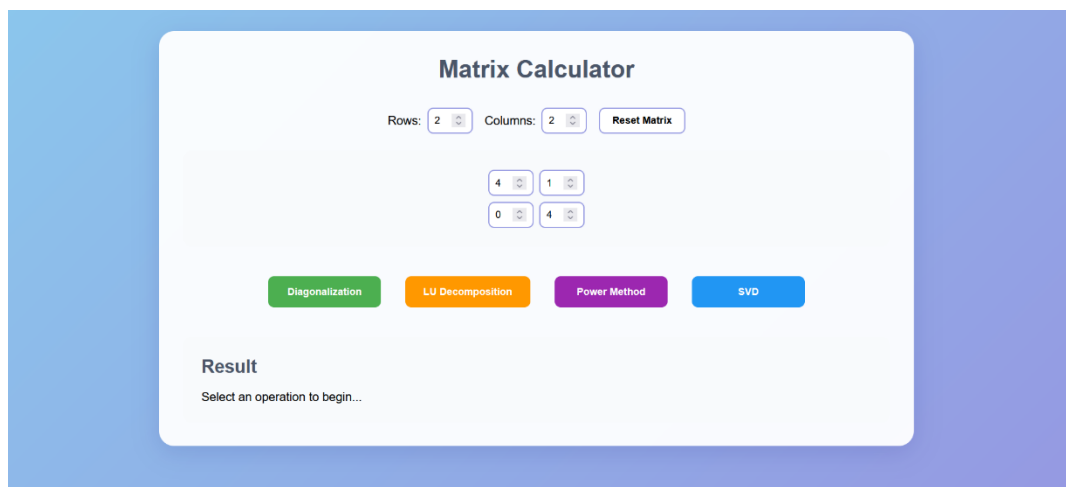
a) Input

- 1) Select the matrix size from the dropdown or type the size you wanted, then click “Reset Matrix”.



The screenshot shows the 'Matrix Calculator' application. At the top, it says 'Matrix Calculator'. Below that, there are input fields for 'Rows' and 'Columns', both set to '2', and a 'Reset Matrix' button. In the center, there is a 2x2 grid of input fields for matrix elements, all containing '0'. Below the grid are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). At the bottom, there is a 'Result' section with the text 'Select an operation to begin...'.

- 2) Enter the matrix elements.



The screenshot shows the 'Matrix Calculator' application with the matrix elements updated. The 'Rows' and 'Columns' fields remain at '2', and the 'Reset Matrix' button is still present. The 2x2 grid of input fields now contains the values 4, 1, 0, and 4. The buttons 'Diagonalization', 'LU Decomposition', 'Power Method', and 'SVD' are still visible. The 'Result' section at the bottom still shows 'Select an operation to begin...'.

- 3) Choose the operation we would like to perform. For this case, choose “LU Decomposition”.
- 4) The result will shown.

b) Output

For this test case, we use the same matrix as before, which is $\begin{bmatrix} 4.0 & 1.0 \\ 0.0 & 4.0 \end{bmatrix}$.

The screenshot shows a web application titled "Matrix Calculator". It has input fields for "Rows" (set to 2) and "Columns" (set to 2), and a "Reset Matrix" button. Below these are four input fields for matrix elements: 4, 1, 0, and 4. At the bottom, there are four buttons: "Diagonalization" (green), "LU Decomposition" (orange), "Power Method" (purple), and "SVD" (blue). The "LU Decomposition" button is highlighted. Below the buttons, the "Result" section displays "LU Decomposition Results". It shows the L Matrix as $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.0000 & 1.0000 \end{bmatrix}$ and the U Matrix as $\begin{bmatrix} 4.0000 & 1.0000 \\ 0.0000 & 4.0000 \end{bmatrix}$.

The result shows that:

- 1) L Matrix: $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.0000 & 1.0000 \end{bmatrix}$

Means that the lower triangular matrix in this case is $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.0000 & 1.0000 \end{bmatrix}$.

- 2) U Matrix: $\begin{bmatrix} 4.0000 & 1.0000 \\ 0.0000 & 4.0000 \end{bmatrix}$

Means that the upper triangular matrix in this case is $\begin{bmatrix} 4.0000 & 1.0000 \\ 0.0000 & 4.0000 \end{bmatrix}$

We use another test case that is singular matrix, which is $\begin{bmatrix} 3.0 & 12.0 \\ 2.0 & 8.0 \end{bmatrix}$.

The screenshot shows the same "Matrix Calculator" interface. The "Rows" and "Columns" are set to 2. The matrix elements are 3, 12, 2, and 8. The "LU Decomposition" button is highlighted. The "Result" section displays "LU Decomposition Results". It shows the L Matrix as $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.6667 & 1.0000 \end{bmatrix}$ and the U Matrix as $\begin{bmatrix} 3.0000 & 12.0000 \\ 0.0000 & 0.0000 \end{bmatrix}$.

The result shows that:

- 1) L Matrix: $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.6667 & 1.0000 \end{bmatrix}$

Means that the lower triangular matrix in this case is $\begin{bmatrix} 1.0000 & 0.0000 \\ 0.6667 & 1.0000 \end{bmatrix}$

2) U Matrix: $\begin{bmatrix} 3.0000 & 12.0000 \\ 0.0000 & 0.0000 \end{bmatrix}$

Means that the upper triangular matrix in this case is $\begin{bmatrix} 3.0000 & 12.0000 \\ 0.0000 & 0.0000 \end{bmatrix}$.

B. Explanation

a) LU Decomposition

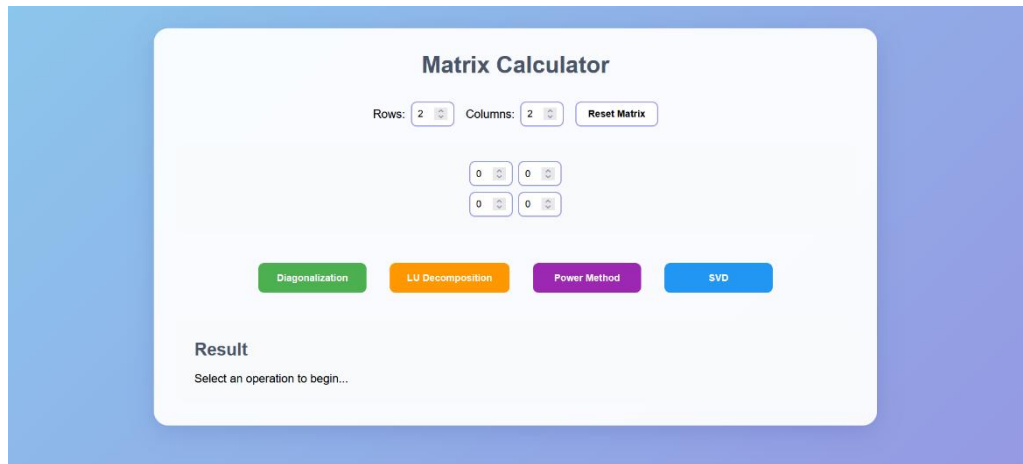
A matrix A can be decomposed into the product of two matrices, which is $A=LU$, where L is a lower triangular matrix with 1s on the diagonal, and U is an upper triangular matrix. LU decomposition could be used in solving systems of linear equations, inverting matrices, and computing determinants.

3. Power Method

A. How to Use the Application and Test Case

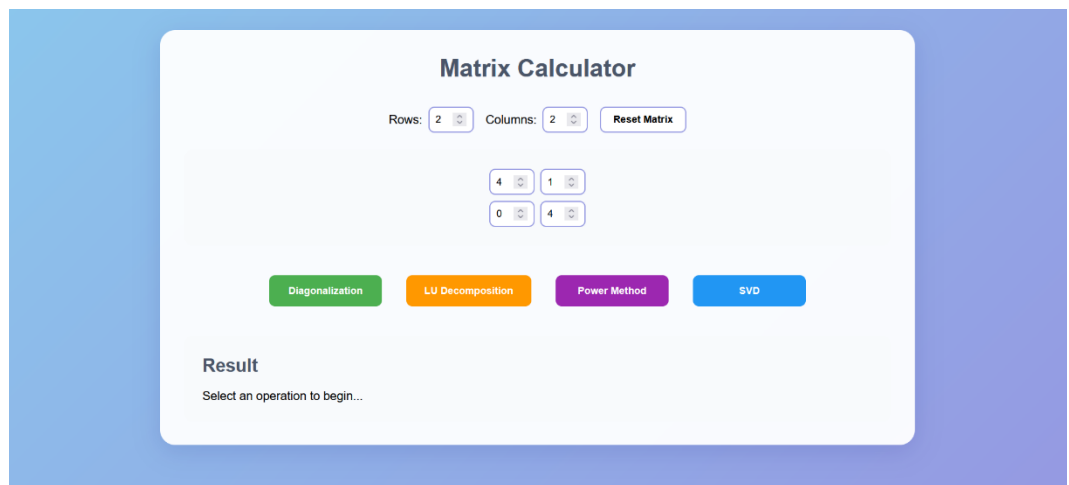
a) Input

- 1) Select the matrix size from the dropdown or type the size you wanted, then click “Reset Matrix”.



The screenshot shows the 'Matrix Calculator' interface. At the top, it says 'Matrix Calculator'. Below that, there are input fields for 'Rows' and 'Columns', both set to '2', and a 'Reset Matrix' button. In the center, there is a 2x2 grid of input fields for matrix elements, all containing '0'. Below the grid are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). At the bottom, there is a 'Result' section with the text 'Select an operation to begin...'.

- 2) Enter the matrix elements.



The screenshot shows the 'Matrix Calculator' interface with the matrix elements updated. The 'Rows' and 'Columns' fields remain at '2', and the 'Reset Matrix' button is still present. The 2x2 grid of input fields now contains the values 4, 1, 0, and 4. The buttons for 'Diagonalization', 'LU Decomposition', 'Power Method', and 'SVD' are still visible. The 'Result' section at the bottom still shows 'Select an operation to begin...'.

- 3) Choose the operation we would like to perform. For this case, choose “Power Method”.
- 4) The result will shown.

b) Output

The screenshot shows a web application titled "Matrix Calculator". It has input fields for "Rows" (set to 2) and "Columns" (set to 2), and a "Reset Matrix" button. Below these are four input fields for matrix elements: 4, 1, 0, and 4, arranged in a 2x2 grid. At the bottom of the input section are four buttons: "Diagonalization" (green), "LU Decomposition" (orange), "Power Method" (purple), and "SVD" (blue). The "Power Method" button is highlighted. Below the buttons, the "Result" section displays "Power Method Results" with an "Eigenvalue: 4.0384" and an "Eigenvector: 0.9992611746313143, 0.03843312210120439".

For this test case, still the same matrix as before, which is $\begin{bmatrix} 4.0 & 1.0 \\ 0.0 & 4.0 \end{bmatrix}$.

The result shows that:

- 1) Eigenvalue: 4.0384 (repeated/non dominant eigenvalue, proved by check diagonalization in No. 1 (Diagonalizability Check))

This is an approximation of the largest eigenvalue of the matrix by using the Power Method, where the Power Method iteratively approximates the eigenvalue, and after a limited number of iterations (which is 100 for this program). The eigenvalues converged when the difference between the current and the previous estimated eigenvalue is smaller than the tolerance ($|\lambda_k - \lambda_{k-1}| < 1 \times 10^{-6}$).

- 2) Eigenvector: [0.9992611746313143, 0.03843312210120439]

The vector represents an approximation of the dominant eigenvector. This is the eigenvector corresponding to the dominant eigenvalue of 4.0384. The eigenvector converged when the difference between the current and the previous estimated eigenvector is smaller than the tolerance ($\|v_k - v_{k-1}\| < 1 \times 10^{-6}$).

Matrix Calculator

Rows: Columns:

5	4
1	2

Result

Power Method Results
 Eigenvalue: 6.0000
 Eigenvector:
 0.9701425001345847, 0.24253562507932197

We use another test case, with the matrix $\begin{bmatrix} 5.0 & 4.0 \\ 1.0 & 2.0 \end{bmatrix}$.

The result shows that:

- 1) Eigenvalue: 6.0000

This is an approximation of the largest eigenvalue of the matrix. Note: In this program, we use the vector of ones for initial guesses.

- 2) Eigenvector: [0.9701425001345847, 0.24253562507932197]

This is the eigenvector corresponding to the dominant eigenvalue

B. Explanation

a) Power Method

Power Method is an iterative method to find the dominant eigenvalue (the largest eigenvalue in magnitude) of a matrix and its corresponding eigenvector. Starting with an initial guess (we use vector of ones here), the method repeatedly applies the matrix to the vector, normalizing it after each multiplication until it converges to the dominant eigenvector.

b) Convergence Criteria

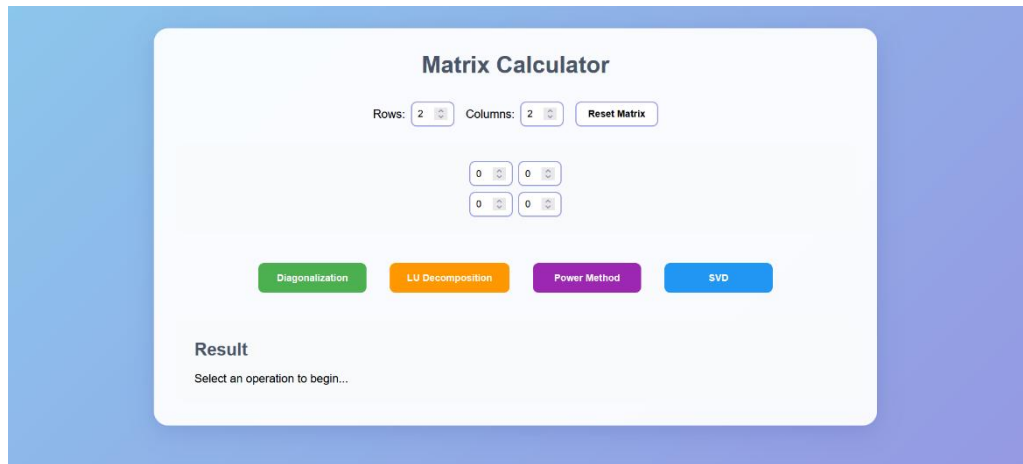
The Power Method converges if the matrix has a dominant eigenvalue with a larger magnitude than the other eigenvalues. The matrix must have a single dominant eigenvalue that is larger in magnitude than all the other eigenvalues. The convergence of the Power Method is typically considered when the difference between the current and the previous estimated eigenvalue is smaller than the tolerance $|\lambda_k - \lambda_{k-1}| < \epsilon$, and when the difference between the current and the previous estimated eigenvector is smaller than the tolerance $\|v_k - v_{k-1}\| < \epsilon$, where ϵ is the tolerance.

4. SVD (Singular Value Decomposition)

A. How to Use the Application and Test Case

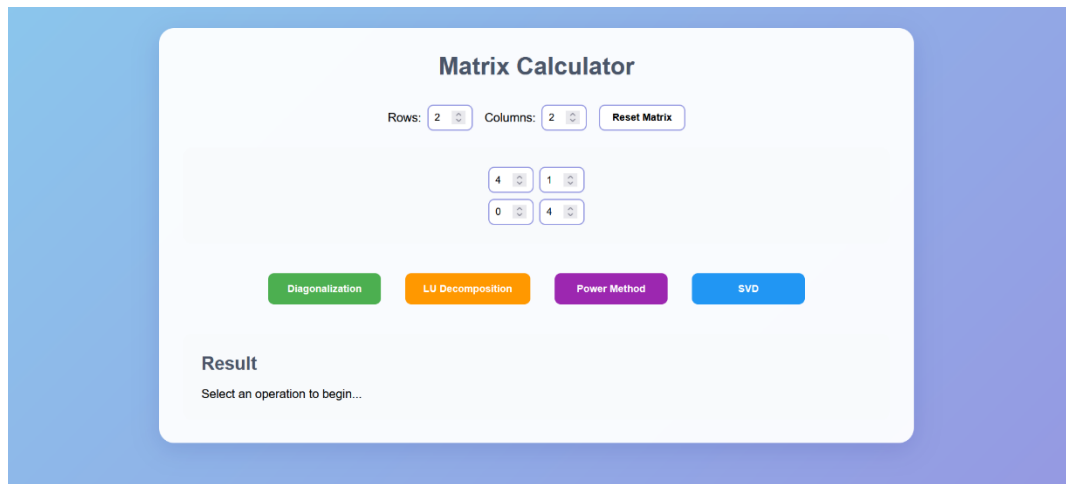
a) Input

- 1) Select the matrix size from the dropdown or type the size you wanted, then click “Reset Matrix”.



The screenshot shows the 'Matrix Calculator' interface. At the top, it says 'Matrix Calculator'. Below that, there are input fields for 'Rows' and 'Columns', both set to '2', and a 'Reset Matrix' button. In the center, there is a 2x2 grid of input fields for matrix elements, all containing '0'. Below the grid are four buttons: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). At the bottom, there is a 'Result' section with the text 'Select an operation to begin...'.

- 2) Enter the matrix elements.



The screenshot shows the 'Matrix Calculator' interface with the matrix elements updated. The 'Rows' and 'Columns' fields remain at '2', and the 'Reset Matrix' button is still present. The 2x2 grid of input fields now contains the values 4, 1, 0, and 4. The buttons for 'Diagonalization', 'LU Decomposition', 'Power Method', and 'SVD' are still visible. The 'Result' section at the bottom still shows 'Select an operation to begin...'.

- 3) Choose the operation we would like to perform. For this case, choose “SVD”.
- 4) The result will shown.

b) Output

Still using the same test case, which is $\begin{bmatrix} 4.0 & 1.0 \\ 0.0 & 4.0 \end{bmatrix}$.

The screenshot shows a web-based 'Matrix Calculator' interface. At the top, there are input fields for 'Rows' (set to 2) and 'Columns' (set to 2), along with a 'Reset Matrix' button. Below these are four input fields for matrix elements: 4, 1, 0, and 4. A row of four buttons is visible: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). The 'SVD' button is highlighted. Under the 'Result' section, the 'SVD Results' are displayed:

SVD Results
U Matrix:
 $\begin{bmatrix} 0.7497 & -0.6618 \\ 0.6618 & 0.7497 \end{bmatrix}$
Singular Values:
4.531128874149275, 3.531128874149275
V^T Matrix:
 $\begin{bmatrix} 0.6618 & 0.7497 \\ -0.7497 & 0.6618 \end{bmatrix}$

The result shows that:

- 1) U Matrix: $\begin{bmatrix} 0.7497 & -0.6618 \\ 0.6618 & 0.7497 \end{bmatrix}$.

These are the eigenvectors of $A A^T$. The columns of U are the left singular vectors of the matrix A, which form an orthonormal basis for the column space of A. The first column of U, $[0.7497, -0.6618]$, is the first left singular vector, while the second column, $[0.6618, 0.7497]$, is the second left singular vector.

- 2) Singular Values: 4.531128874149275, 3.531128874149275

The diagonal elements of the matrix Σ are the singular values, which are always non-negative. These values represent the "strength" or "magnitude" of the corresponding singular vectors. These singular values are the square roots of the eigenvalues of $A^T A$ or $A A^T$. The singular values indicate how much the matrix stretches space in the direction of each singular vector. Larger singular values correspond to stronger contributions in the transformation.

- 3) V^T Matrix: $\begin{bmatrix} 0.6618 & 0.7497 \\ -0.7497 & 0.6618 \end{bmatrix}$

These are the eigenvectors of $A^T A$. The rows of V^T are the right singular vectors of the matrix A, and they form an orthonormal basis for the row space of A. The first row of V^T , $[0.6618, 0.7497]$, is the first right singular vector, while the second row of V^T , $[-0.7497, 0.6618]$, is the second right singular vector.

We use another matrix, which is $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ (matrix with zero singular values)

The screenshot shows a web-based 'Matrix Calculator' interface. At the top, it has input fields for 'Rows' (set to 2) and 'Columns' (set to 2), along with a 'Reset Matrix' button. Below these are input fields for the matrix elements: row 1 contains 1 and 2, and row 2 contains 2 and 4. There are four buttons for different matrix operations: 'Diagonalization' (green), 'LU Decomposition' (orange), 'Power Method' (purple), and 'SVD' (blue). The 'SVD' button is highlighted. Under the 'Result' section, the 'SVD Results' are displayed:

SVD Results
U Matrix:
[-0.4472 -0.8944]
[-0.8944 0.4472]
Singular Values:
5.000000000000001, 1.9860273225978183e-16
V^T Matrix:
[-0.4472 -0.8944]
[-0.8944 0.4472]

The result shows that:

- 1) U Matrix: $\begin{bmatrix} -0.4472 & -0.8944 \\ -0.8944 & 0.4472 \end{bmatrix}$.

The first column of U, $[-0.8944, 0.4472]$, is the first left singular vector, while the second column, $[0.6618, 0.7497]$, is the second left singular vector.

- 2) Singular Values: 5.000000000000001, 1.9860273225978183e-16

The first singular value represents the magnitude of the primary direction of the data in the matrix, which is nearly 5, while the second singular value being close to zero indicates that the second direction in the matrix is essentially "redundant" or linearly dependent on the first direction.

- 3) V^T Matrix: $\begin{bmatrix} -0.4472 & -0.8944 \\ -0.8944 & 0.4472 \end{bmatrix}$

The first row of V^T, $[-0.4472, -0.8944]$, is the first right singular vector, while the second row of V^T, $[-0.8944, 0.4472]$, is the second right singular vector.

Using another matrix, which is $\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$

The screenshot shows the same 'Matrix Calculator' interface, but with 'Rows' set to 3 and 'Columns' set to 2. The matrix input fields now contain: row 1 (1, 2), row 2 (2, 4), and row 3 (3, 6). The 'SVD' button remains highlighted. The 'Result' section shows the 'SVD Results' for this 3x2 matrix:

SVD Results
U Matrix:
[-0.2673 0.9562 0.1195]
[-0.5345 -0.0439 -0.8440]
[-0.8018 -0.2895 0.5228]
Singular Values:
8.366600265340756, 8.881784197001252e-16
V^T Matrix:
[-0.4472 -0.8944]
[-0.8944 0.4472]

The result shows that:

- 1) U Matrix: $\begin{bmatrix} -0.2673 & 0.9562 & 0.1195 \\ -0.5345 & -0.0439 & -0.8440 \\ -0.8018 & -0.2895 & 0.5228 \end{bmatrix}$.

The each column corresponds to an orthonormal basis for the column space of the matrix.

- 2) Singular Values: 8.366600265340756, $8.881784197001252 \times 10^{-16}$

The first singular value is around 8.3666, which is large and indicates that the data in the matrix has significant variation along the direction of the first singular vector. The second singular value is extremely small ($8.881784197001252 \times 10^{-16}$), which is effectively zero), suggesting that the second direction in the data is nearly redundant, meaning it is almost perfectly linearly dependent on the first direction. This is because the rows are linearly dependent, reducing the rank of the matrix.

- 3) V^T Matrix: $\begin{bmatrix} -0.4472 & -0.8944 \\ -0.8944 & 0.4472 \end{bmatrix}$

The first row of V^T , $[-0.4472, -0.8944]$, is the first right singular vector, while the second row of V^T , $[-0.8944, 0.4472]$, is the second right singular vector.

B. Explanation

a) SVD

Any matrix A can be decomposed as $A=U\Sigma V^T$, where U and V are orthogonal matrices, and Σ is a diagonal matrix with non-negative real numbers (singular values). SVD is usually used in signal processing, statistics, and machine learning for dimensionality reduction, least-squares fitting, etc.

Documentation for Developers

A. Back-End

- 1) **Import all libraries and frameworks** needed to create functions and applications.

```
from flask import Flask, render_template, request, jsonify
import numpy as np
from scipy.linalg import lu
from numpy.linalg import svd, eig

app = Flask(__name__)
```

- 2) **The Diagonalization function** is used to check if a matrix is diagonalizable, as well as the matrix's eigenvalues and eigenvectors. Function will return Boolean for diagonalizable and the result of eigenvalues and eigenvectors.

```
def check_diagonalizability(matrix):
    eigenvalues, eigenvectors = np.linalg.eig(matrix)
    unique_eigenvalues = np.unique(eigenvalues)
    for eigenvalue in unique_eigenvalues:
        am = np.count_nonzero(np.isclose(eigenvalues, eigenvalue))
        gm = np.linalg.matrix_rank(matrix - eigenvalue * np.eye(matrix.shape[0]))
        gm = matrix.shape[0] - gm
        if am != gm:
            return False, eigenvalues, eigenvectors
    return True, eigenvalues, eigenvectors
```

- 3) **The LU Decomposition function** is used to find the L and U values of the matrix. L is the result of the lower triangular matrix while U is the result of the upper triangular matrix.

```

def lu_decomposition(matrix):
    n = matrix.shape[0]
    L = np.zeros((n, n))
    U = np.zeros((n, n))

    for i in range(n):
        # Upper Triangular
        for j in range(i, n):
            U[i][j] = matrix[i][j] - sum(L[i][k] * U[k][j] for k in range(i))

        # Lower Triangular
        for j in range(i, n):
            if i == j:
                L[i][i] = 1 # Diagonal as 1
            else:
                L[j][i] = (matrix[j][i] - sum(L[j][k] * U[k][i] for k in range(i))) / U[i][i]

    return L, U

```

- 4) **The Singular Value Decomposition (SVD) function** returns U which is the Matrix of left singular vectors. S/ Σ (Sigma) which is the Diagonal matrix of singular values. VT (V Transpose) is the Matrix of right singular vectors.

```

def singular_value_decomposition(matrix):
    U, S, VT = svd(matrix)
    return U, S, VT

```

- 5) **Power Method function** used to determine the dominant eigenvalue (eigenvalue with the largest absolute value) and associated eigenvector of a matrix.

```

def power_method(matrix, tol=1e-6, max_iter=100):
    n = len(matrix)
    vector = np.ones(n)

    eigenvalue = 0
    for i in range(max_iter):
        new_vector = np.dot(matrix, vector)
        new_vector_norm = np.linalg.norm(new_vector)
        new_vector = new_vector / new_vector_norm # Normalize the new vector
        new_eigenvalue = np.dot(new_vector, np.dot(matrix, new_vector)) / np.dot(new_vector, new_vector)
        if abs(new_eigenvalue - eigenvalue) < tol:
            break
        eigenvalue = new_eigenvalue
        vector = new_vector

    return eigenvalue, vector

```


- 6) **Index Function** used to connect the result we want to show at front-end and the function we have made before at back-end.

```
@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        try:
            data = request.get_json()
            if not data:
                return jsonify({"error": "No data received"}), 400

            matrix_data = data.get('matrix')
            rows = data.get('rows')
            cols = data.get('cols')
            operation = data.get('operation')

            if not all([matrix_data, rows, cols, operation]):
                return jsonify({"error": "Missing required data"}), 400

            matrix = np.array(matrix_data).reshape(rows, cols).astype(float)
            result = None

            if operation in ["diagonalization", "lu_decomposition", "power_method"] and rows != cols:
                return jsonify({"error": "Matrix must be square for this operation"}), 400

            if operation == "diagonalization":
                is_diagonalizable, eigenvalues, eigenvectors = check_diagonalizability(matrix)
                result = {
                    "operation": "diagonalization",
                    "is_diagonalizable": bool(is_diagonalizable),
                    "eigenvalues": eigenvalues.tolist(),
                    "eigenvectors": eigenvectors.tolist(),
                    "input_matrix": matrix.tolist()
                }

            elif operation == "lu_decomposition":
                L, U = lu_decomposition(matrix)
                result = {
                    "operation": "lu_decomposition",
                    "input_matrix": matrix.tolist(),
                    "L": L.tolist(),
                    "U": U.tolist()
                }

            elif operation == "svd":
                U, S, VT = singular_value_decomposition(matrix)
                result = {
                    "operation": "svd",
                    "input_matrix": matrix.tolist(),
                    "U": U.tolist(),
                    "S": S.tolist(),
                    "VT": VT.tolist()
                }

            elif operation == "power_method":
                eigenvalue, eigenvector = power_method(matrix)
                result = {
                    "operation": "power_method",
                    "input_matrix": matrix.tolist(),
                    "eigenvalue": float(eigenvalue),
                    "eigenvector": eigenvector.tolist()
                }

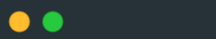
            else:
                return jsonify({"error": f"Unknown operation: {operation}"}), 400

            if result is not None:
                return jsonify(result)
            else:
                return jsonify({"error": "Operation failed"}), 400

        except Exception as e:
            return jsonify({"error": str(e)}), 400

    return render_template("index.html")
```

- 7) In a Flask application, this is usually used to **launch the Flask development server**. This line of code is commonly used in Python scripts to find out if a Python file is being run as the main program or if it's being imported as a module in another script. This condition is True when the script is executed directly, and the code block under this condition will execute.



```
if __name__ == "__main__":
    app.run(debug=True)
```

B. Developer (Front-End)

*Comment included inside the code snippet

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Matrix Calculator</title>
  <style>
    /* Global style resets and font settings */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: Arial, sans-serif;
    }
    /* Background and layout for the body */
    body {
      background: #f0f2f5;
      min-height: 100vh;
      padding: 2rem;
      background: linear-gradient(135deg, #B0C4DE 0%, #9599E2 100%);
    }
    /* Loading message styling */
    .loading {
      display: none; /* Hidden by default */
      text-align: center;
      padding: 20px;
      font-weight: bold;
      color: #4a5568;
    }
    .loading.show {
      display: block; /* Show when needed */
    }
    /* Main calculator container styling */
    .calculator-container {
      max-width: 300px;
      margin: 0 auto;
      background: rgba(255, 255, 255, 0.95);
      border-radius: 20px;
      padding: 2rem;
      box-shadow: 0 8px 32px rgba(31, 38, 135, 0.15);
    }
    /* Title styling */
    .title {
      text-align: center;
      color: #4a5568;
      margin-bottom: 2rem;
      font-size: 2rem;
    }
    /* Controls for matrix dimensions */
    .matrix-controls {
      display: flex;
      gap: 1rem; /* Space between controls */
      margin-bottom: 1.5rem;
      align-items: center; /* Center vertically */
      justify-content: center; /* Center horizontally */
    }
    .dimension-control {
      display: flex;
      align-items: center;
      gap: 0.5rem; /* Space between label and input */
    }
    input[type="number"] {
      width: 60px;
      padding: 0.5rem;
      border: 2px solid #9599E2;
      border-radius: 8px;
      outline: none;
      transition: all 0.3s ease; /* Smooth transition for focus */
    }
  </style>
</head>

```

```

/* Matrix input area styling */
.matrix-input {
  margin-bottom: 2rem;
  padding: 1.5rem;
  background: #f8fafc;
  border-radius: 12px;
}
.matrix-grid {
  display: flex;
  flex-direction: column;
  gap: 0.5rem; /* Space between rows */
  align-items: center;
}
.matrix-row {
  display: flex;
  gap: 0.5rem; /* Space between inputs in a row */
}
/* Operation button styling */
.operation-btn {
  padding: 0.8rem 1.5rem;
  border: none;
  border-radius: 8px;
  color: white;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.3s ease;
  margin: 0.5rem;
  min-width: 150px; /* Minimum button width */
}
.operation-btn:hover {
  transform: translateY(-2px); /* Lift effect on hover */
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15); /* Shadow effect on hover */
}
/* Specific button colors for different operations */
.diagonalization { background: #4CAF50; }
.svd { background: #2196F3; }
.power { background: #9C27B0; }
.lu { background: #FF9800; }

.operations {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 1rem;
  margin-bottom: 2rem;
}
/* Result container styling */
.result-container {
  background: #f8fafc;
  padding: 1.5rem;
  border-radius: 12px;
  margin-top: 2rem;
}
.result-title {
  color: #4a5568;
  margin-bottom: 1rem;
}
/* Display formatting for matrices and results */
.matrix-display {
  font-family: monospace;
  white-space: pre;
  overflow-x: auto;
  padding: 1rem;
  background: #ffff;
  border-radius: 8px;
  margin: 1rem auto; /* Centering the matrix display */
  border: 1px solid #e2e8f0; /* Border for matrix display area */
}
/* Error message styling */
.error-message {
  color: #dc3545; /* Red text for errors */
  background: #fee2e2; /* Light red background for errors */
  padding: 1rem;
  border-radius: 8px;
  margin-top: 1rem;
  border: 2px solid #fecaca; /* Border for error message box */
  display: none; /* Hidden by default */
}
.error-message.show {
  display: block; /* Show when needed */
}

```

```

.button-controls {
  display: flex;
  justify-content: center;
  gap: 10px; /* Space between buttons in control area */
  margin-bottom: 10px;
}

.control-btn {
  padding: 0.5rem 1rem;
  border: 2px solid #9599E2;
  border-radius: 8px;
  background-color: white;
  cursor: pointer;
  font-weight: bold;
  transition: all 0.3s ease; /* Smooth transition on hover */
}

.control-btn:hover {
  background-color: #9599E2; /* Change background on hover */
  color: white; /* Change text color on hover */
}

</style>
</head>
<body>
  <div class="calculator-container">
    <h1 class="title">Matrix Calculator</h1>

    <!-- Controls for setting matrix dimensions -->
    <div class="matrix-controls">
      <div class="dimension-control">
        <label>Rows:</label>
        <input type="number" min="1" max="10" value="3" id="matrix-rows"> <!-- Input for number of rows -->
      </div>
      <div class="dimension-control">
        <label>Columns:</label>
        <input type="number" min="1" max="10" value="3" id="matrix-cols"> <!-- Input for number of columns -->
      </div>
      <button class="control-btn" onclick="resetMatrix()">Reset Matrix</button> <!-- Button to reset matrix -->
    </div>

    <!-- Area where the matrix inputs will be displayed -->
    <div class="matrix-input">
      <div class="matrix-grid" id="matrix-grid"></div> <!-- Container for matrix input fields -->
    </div>

    <!-- Error message display area -->
    <div class="error-message" id="error-message"> You need to input a valid matrix for this calculation. </div>

    <!-- Operation buttons for different calculations -->
    <div class="operations">
      <button class="operation-btn diagonalization" onclick="calculate('diagonalization')">Diagonalization</button>
      <button class="operation-btn lu" onclick="calculate('lu.decomposition')">LU Decomposition</button>
      <button class="operation-btn power" onclick="calculate('power_method')">Power Method</button>
      <button class="operation-btn svd" onclick="calculate('svd')">SVD</button>
    </div>

    <!-- Loading indicator during calculations -->
    <div class="loading" id="loading">Calculating...</div>

    <!-- Result container to display calculation results -->
    <div class="result-container">
      <h2 class="result-title">Result</h2>
      <div id="result">Select an operation to begin...</div> <!-- Placeholder for results -->
    </div>
  </div>

```

```

<script>
  let currentRows = 3; // Default number of rows
  let currentCols = 3; // Default number of columns

  // Function to create a matrix grid based on specified dimensions
  function createMatrix(rows, cols) {
    const grid = document.getElementById('matrix-grid');
    grid.innerHTML = ''; // Clear existing inputs
    currentRows = rows; // Update current row count
    currentCols = cols; // Update current column count

    // Create rows and columns of input fields
    for (let i = 0; i < rows; i++) {
      const row = document.createElement('div'); // Create a new row
      row.className = 'matrix-row'; // Assign row class
      for (let j = 0; j < cols; j++) {
        const input = document.createElement('input'); // Create an input field
        input.type = 'number'; // Set input type to number
        input.value = 0; // Default value
        input.step = 'any'; // Allow decimal values
        row.appendChild(input); // Append input to the row
      }
      grid.appendChild(row); // Append the row to the grid
    }
  }

  // Function to retrieve values from the matrix inputs
  function getMatrixValues() {
    const inputs = document.querySelectorAll('.matrix-row input'); // Select all input fields
    return Array.from(inputs).map(input => parseFloat(input.value) || 0); // Convert values to float or default to zero
  }

  // Function to format a matrix into a string representation
  function formatMatrix(matrix, rows, cols) {
    let result = '';
    for (let i = 0; i < rows; i++) {
      result += '['; // Start new row representation
      for (let j = 0; j < cols; j++) {
        result += matrix[i][j].toFixed(4).padStart(10) + ' '; // Format each element with four decimal places
      }
      result += ']\n'; // End row representation
    }
    return result; // Return formatted string
  }

  // Function to show error messages to users
  function showError(message) {
    const errorElement = document.getElementById('error-message');
    errorElement.textContent = message; // Set error message text
    errorElement.classList.add('show'); // Show error message

    setTimeout(() => { // Hide after a delay
      errorElement.classList.remove('show');
    }, 5000);
  }

  // Function to toggle loading indicator visibility
  function showLoading(show) {
    document.getElementById('loading').classList.toggle('show', show); // Show or hide loading indicator based on parameter
  }

```

```

// Main calculation function that handles different operations on the matrix
async function calculate(operation) {
  const rows = currentRows;
  const cols = currentCols;

  const matrix = getMatrixValues(); // Get current matrix values

  if (operation !== 'svd' && rows !== cols) { // Check if operation requires square matrix
    showError('You need to input a valid square matrix for this calculation');
    return;
  }

  showLoading(true); // Show loading indicator while processing

  document.getElementById('result').innerHTML = ''; // Clear previous results

  try {
    const response = await fetch('/', { // Send request to server with matrix data
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body:
        JSON.stringify({ matrix, rows, cols, operation }) // Send data as JSON string
    });

    const data = await response.json(); // Parse JSON response

    if (data.error) { // Handle server errors
      showError(data.error);
      return;
    }

    let resultHtml = '';

    if (data.operation === 'svd') { // Handle SVD operation results
      resultHtml += '<h3>SVD Results</h3>';
      resultHtml += '<pre>U Matrix:\n${formatMatrix(data.U, rows, rows)}</pre>';
      resultHtml += '<pre>Singular Values:\n${data.S.join(', ')}</pre>';
      resultHtml += '<pre>V^T Matrix:\n${formatMatrix(data.VT, cols, cols)}</pre>';
    } else if (data.operation === 'diagonalization') { // Handle Diagonalization results
      resultHtml += '<h3>Diagonalization Results</h3>';
      resultHtml += '<p>Diagonalizable: ${data.is_diagonalizable}</p>';
      resultHtml += '<pre>Eigenvalues:\n${data.eigenvalues.join(', ')}</pre>';
      resultHtml += '<pre>Eigenvectors:\n${formatMatrix(data.eigenvectors, rows, rows)}</pre>';
    } else if (data.operation === 'lu_decomposition') { // Handle LU Decomposition results
      resultHtml += '<h3>LU Decomposition Results</h3>';
      resultHtml += '<pre>L Matrix:\n${formatMatrix(data.L, rows, rows)}</pre>';
      resultHtml += '<pre>U Matrix:\n${formatMatrix(data.U, rows, rows)}</pre>';
    } else if (data.operation === 'power_method') { // Handle Power Method results
      resultHtml += '<h3>Power Method Results</h3>';
      resultHtml += '<p>Eigenvalue: ${data.eigenvalue.toFixed(4)}</p>';
      resultHtml += '<pre>Eigenvector:\n${data.eigenvector.join(', ')}</pre>';
    }

    document.getElementById('result').innerHTML = resultHtml;

  } catch (error) { // Handle fetch errors
    showError('Calculation failed: ' + error.message);
  } finally {
    showLoading(false); // Hide loading indicator after processing is complete
  }
}

// Function to reset the matrix based on user-defined dimensions
function resetMatrix() {
  const rows = parseInt(document.getElementById('matrix-rows').value); // Get number of rows from input field
  const cols = parseInt(document.getElementById('matrix-cols').value); // Get number of columns from input field

  createMatrix(rows, cols); // Recreate the matrix with new dimensions
}

// Initialize the matrix grid on page load with default size of 3x3.
createMatrix(3,3);
</script>
</body>
</html>

```